

Guía de instalación Agente Wazuh en Alpine Linux

1 INTRODUCCIÓN

Esta guía documenta cómo compilar, instalar y registrar el agente de Wazuh 4.12.0 desde código fuente sobre Alpine Linux 3.21.5.

El servidor web implementado por la organización fue hecho con Alpine Linux 3.21.5, una distribución que usa `musl` como implementación de la librería estándar de C. Esto hace que Wazuh no dé soporte oficial a Alpine ni a `musl`. El propio equipo de Wazuh tiene un pedido abierto de la comunidad desde 2020 solicitando este soporte, sin resolverlo de forma oficial hasta la fecha. Todo lo que sigue en esta guía es una solución para el contexto del laboratorio 7, por lo que no es oficial y no hay restricción si el grupo quiere implementar otra forma de instalación del agente.

2 EL PROBLEMA

`musl` y `glibc` son ambas implementaciones de la librería estándar de C, pero no son binariamente compatibles y difieren en varios puntos clave. Wazuh fue escrito asumiendo `glibc`, por lo que da por sentados los siguientes aspectos:

- **Extensiones GNU que `musl` no implementa:** funciones como `random_r`, `gnu_dev_major`, `mkstemp64`, `backtrace`, o toda la familia "LFS64" (`open64`, `stat64`, etc.) existen en `glibc` pero simplemente no existen en `musl`.
- **Funciones retiradas por decisión de diseño:** `getcontext`, `setcontext` y `makecontext` (manejo de contexto de ejecución) fueron eliminadas de POSIX y `musl` decidió no implementarlas nunca.
- **Comportamiento distinto bajo las mismas macros:** el mismo código fuente, compilado con las mismas flags, se comporta distinto en cada sistema; por ejemplo, `strerror_r`.
- **Herramientas de sistema distintas:** Alpine usa BusyBox en vez de las utilidades

GNU/Coreutils. Por ejemplo, su `ps` no soporta el flag `-p` de la misma forma, lo cual rompe scripts de Wazuh que dan por sentado un `ps` estilo GNU.

A esto se suma que el agente de Wazuh 4.12, usado por la administración en su manager y dashboard, incorpora monitoreo basado en eBPF (módulo FIM/whodata), que trae sus propias dependencias específicas (Clang, libbpf, bpftool) y sus propios problemas de compilación en un entorno no probado por el equipo de Wazuh.

3 CONCEPTOS PREVIOS

Antes de seguir los pasos, vale la pena entender por qué hace falta cada cosa que se va a realizar (no solo copiar y pegar comandos). Esta sección explica los conceptos que aparecen una y otra vez en la guía.

3.1 ¿Qué es musl y por qué existe junto a glibc?

Todo programa en C necesita una librería estándar de C (libc): es la capa que traduce funciones básicas como abrir un archivo, pedir memoria o formatear un texto en llamadas reales al sistema operativo. Ningún programa en C corre sin una.

- **glibc (GNU C Library):** es la libc que usan la gran mayoría de distribuciones Linux "grandes": Ubuntu, Debian, CentOS y Fedora. Es la más completa, pero también la más pesada.
- **musl:** es una libc alternativa, escrita para ser pequeña, simple y rápida. Alpine Linux la eligió como su libc por defecto precisamente por eso; es una de las razones por las que las imágenes de Alpine pesan tan poco comparadas con otras distribuciones.

3.2 ¿Qué es make y por qué se compila con él?

`make` es una herramienta que automatiza la compilación de proyectos grandes con muchos archivos fuente. En vez de compilar archivo por archivo a mano, un archivo llamado Makefile describe las reglas ("para construir X, primero hay que tener Y y Z, y correr este comando") y `make` se encarga de ejecutar todo en el orden correcto.

Wazuh usa un Makefile gigante que compila cientos de archivos fuente propios más docenas de librerías de terceros (`src/external`). Por ejemplo:

```
make TARGET=agent CFLAGS="..."
```

3.3 ¿Qué es eBPF y por qué aparece en esta guía?

eBPF (*extended Berkeley Packet Filter*) es una tecnología del kernel de Linux que permite ejecutar pequeños programas dentro del kernel mismo, de forma segura, para observar eventos del sistema (llamadas al sistema, acceso a archivos, tráfico de red) sin necesidad de escribir un módulo de kernel tradicional. El módulo FIM/whodata de Wazuh 4.12 lo usa para detectar en tiempo real qué proceso modificó un archivo.

Estos programas eBPF se compilan con Clang usando una técnica llamada **CO-RE** (*Compile Once – Run Everywhere*): el binario resultante puede ejecutarse en kernels con estructuras internas ligeramente distintas a la del kernel donde se compiló, gracias a información de depuración (BTF) que el kernel lee en tiempo de carga. Para esto, Clang necesita macros y estructuras específicas del kernel (`__builtin_preserve_access_index`, tipos de mapas BPF, etc.) que en un sistema glibc normalmente llegan con el paquete `linux-headers`, pero que en Alpine vienen incompletos — de ahí las cabeceras de compatibilidad que se crearán en el Paso 2.

3.4 ¿Qué son los sub-builds de CMake dentro del Makefile de Wazuh?

No todo el código de Wazuh se compila con `make`. Dos componentes — `sysinfo/data_provider` (recolección de información del sistema) y `syscollector` (inventario de hardware/software) — son proyectos independientes que usan **CMake**, otra herramienta de generación de builds, más común en proyectos C++. El Makefile principal de Wazuh, al llegar a esos componentes, invoca a CMake como un proceso hijo para que los compile.

Esto importa para la guía porque CMake **no hereda automáticamente** las variables de entorno de compilación (como `LDFLAGS`) de la forma en que lo hace `make`; cualquier flag de enlace que dependa la compatibilidad con `musl` hay que pasárselo explícitamente vía la variable `CMAKE_OPTS`, como se hará en el Paso 7.

4 REQUISITOS PREVIOS

Antes de empezar, debe confirmarse lo siguiente:

- **Acceso root** en el contenedor o máquina Alpine donde se va a compilar el agente. Todos los comandos de esta guía asumen que se ejecutan como `root`.
- **Arquitectura x86_64**. El Paso 5.2 (`libucontext`) compila explícitamente con `ARCH=x86_64`. Si el servidor Alpine corre en `aarch64` (por ejemplo, un Mac con Apple Silicon o

una instancia ARM en la nube), ese valor debe cambiarse por `ARCH=arm64` antes de compilar, o el agente no arrancará.

- **Conectividad de salida a internet** desde el contenedor, para `git clone`, `apk add` y `make deps` (que descarga las dependencias de `src/external`).
- **Conectividad de red hacia el Wazuh manager** en los puertos **1515/TCP** (registro, Paso 9) y **1514/TCP** (conexión persistente, Paso 10) — conviene verificarla antes de empezar a compilar, no solo al final.
- **Tiempo:** la compilación completa (Pasos 3 a 7) puede tomar entre 20 y 40 minutos dependiendo de los recursos del contenedor, en su mayoría por OpenSSL y los sub-builds de CMake.

5 SOLUCIÓN

5.1 Paso 1 — Paquetes de desarrollo en Alpine

```
apk update && apk add --no-cache \  
  build-base cmake git curl unzip tar automake autoconf libtool pkgconfig \  
  linux-headers musl-dev bsd-compat-headers gettext-dev \  
  clang llvm zlib-dev elfutils-dev
```

`clang`, `llvm`, `zlib-dev` y `elfutils-dev` son necesarios porque el agente 4.12 incluye monitoreo con eBPF (módulo FIM/whodata), que requiere compilar programas BPF con Clang y enlazar contra `libbpf`.

5.2 Paso 2 — Clonar Wazuh 4.12.0 y crear cabeceras de compatibilidad musl/BPF

```
rm -rf /root/wazuh  
git clone --depth 1 --branch v4.12.0 https://github.com/wazuh/wazuh.git /root/wazuh  
  
mkdir -p /usr/include/linux /usr/include/bpf  
  
cat << 'EOF' > /usr/include/linux/compiler.h  
#ifndef _LINUX_COMPILER_H  
#define _LINUX_COMPILER_H
```

```

#include <linux/bpf.h>
#ifndef __user
#define __user
#endif
#ifndef __force
#define __force
#endif
#ifndef __must_check
#define __must_check
#endif
#ifndef __always_inline
#define __always_inline inline
#endif
#ifndef __noreturn
#define __noreturn __attribute__((noreturn))
#endif
#ifndef __weak
#define __weak __attribute__((weak))
#endif
#ifndef __packed
#define __packed __attribute__((packed))
#endif
#ifndef __aligned
#define __aligned(x) __attribute__((aligned(x)))
#endif
#ifndef __printf
#define __printf(a, b) __attribute__((format(printf, a, b)))
#endif
#ifndef __maybe_unused
#define __maybe_unused __attribute__((unused))
#endif
#ifndef __always_unused
#define __always_unused __attribute__((unused))
#endif
#ifndef fallthrough
#define fallthrough __attribute__((fallthrough))
#endif
#ifndef BPF_REG_0
#define BPF_REG_0 0

```

```

#endif
#ifndef BPF_MOV64_IMM
#define BPF_MOV64_IMM(DST, IMM) \
    ((struct bpf_insn) { \
        .code = 0xb7, \
        .dst_reg = DST, \
        .src_reg = 0, \
        .off = 0, \
        .imm = IMM })
#endif
#ifndef BPF_EXIT_INSN
#define BPF_EXIT_INSN() \
    ((struct bpf_insn) { \
        .code = 0x95, \
        .dst_reg = 0, \
        .src_reg = 0, \
        .off = 0, \
        .imm = 0 })
#endif
#endif
EOF

cat << 'EOF' > /usr/include/linux/kernel.h
#ifndef _LINUX_KERNEL_H
#define _LINUX_KERNEL_H
#include <sys/param.h>
#ifndef ARRAY_SIZE
#define ARRAY_SIZE(arr) (sizeof(arr) / sizeof((arr)[0]))
#endif
#ifndef roundup
#define roundup(x, y) (((x) + ((y) - 1)) / (y)) * (y)
#endif
#endif
EOF

cat << 'EOF' > /usr/include/linux/err.h
#ifndef _LINUX_ERR_H
#define _LINUX_ERR_H
#include <linux/compiler.h>

```

```

#include <errno.h>
#include <stdbool.h>
#define MAX_ERRNO 4095
#define IS_ERR_VALUE(x) ((unsigned long)(void *) (x) >= (unsigned long)-MAX_ERRNO)
static inline void *ERR_PTR(long error) { return (void *) error; }
static inline long PTR_ERR(const void *ptr) { return (long) ptr; }
static inline bool IS_ERR(const void *ptr) { return IS_ERR_VALUE((unsigned long)ptr); }
static inline bool IS_ERR_OR_NULL(const void *ptr) { return !ptr || IS_ERR_VALUE((unsigned long)ptr); }
static inline void *ERR_CAST(const void *ptr) { return (void *) ptr; }
static inline int PTR_ERR_OR_ZERO(const void *ptr) { return IS_ERR(ptr) ? PTR_ERR(ptr) : 0; }
#endif
EOF

cat << 'EOF' > /usr/include/linux/bpf_insn.h
#ifndef _LINUX_BPF_INSN_H
#define _LINUX_BPF_INSN_H
#include <linux/bpf.h>
#ifndef BPF_REG_0
#define BPF_REG_0 0
#endif
#ifndef BPF_MOV64_IMM
#define BPF_MOV64_IMM(DST, IMM) \
    ((struct bpf_insn) { \
        .code = 0xb7, \
        .dst_reg = DST, \
        .src_reg = 0, \
        .off = 0, \
        .imm = IMM })
#endif
#ifndef BPF_EXIT_INSN
#define BPF_EXIT_INSN() \
    ((struct bpf_insn) { \
        .code = 0x95, \
        .dst_reg = 0, \
        .src_reg = 0, \
        .off = 0, \

```

```

    .imm    = 0 })
#endif
#endif
EOF
cp /usr/include/linux/bpf_insn.h /usr/include/bpf/bpf_insn.h

if ! grep -q "BPF_MAP_TYPE_ARENA" /usr/include/linux/bpf.h 2>/dev/null; then
cat << 'EOF' >> /usr/include/linux/bpf.h
#ifdef BPF_MAP_TYPE_ARENA
#define BPF_MAP_TYPE_ARENA 31
#endif
EOF
fi

```

Alpine cuenta con un conjunto reducido de cabeceras del espacio de usuario (*userspace*) del kernel de Linux, por lo que para lograr compilar con éxito el agente de **Wazuh 4.12.0**, es necesario crear cabeceras de compatibilidad personalizadas. A continuación se detalla el propósito técnico de cada una de las cabeceras e inyecciones de código creadas arriba.

5.2.1 `/usr/include/linux/compiler.h`

- **Propósito:** proveer macros de atributos del compilador GCC/Clang y definiciones eBPF base.
- **Explicación:** el código de Wazuh (especialmente los módulos de eBPF) utiliza anotaciones específicas del kernel de Linux como `__user`, `__packed`, `__always_inline` o `fallthrough`. En sistemas GNU tradicionales estas provienen de las cabeceras internas del kernel, pero en musl no están presentes por defecto. Esta cabecera evita errores de sintaxis definiendo estos atributos para el compilador.

5.2.2 `/usr/include/linux/kernel.h`

- **Propósito:** ofrecer macros utilitarias estándar para manipulación de datos y memoria.
- **Explicación:** define utilidades fundamentales a nivel de sistema que el código de Wazuh espera encontrar en las cabeceras del kernel, tales como `ARRAY_SIZE` (utilizada para calcular de manera segura la cantidad de elementos en un arreglo) y `roundup` (empleada en la alineación de bloques de memoria).

5.2.3 `/usr/include/linux/err.h`

- **Propósito:** recrear la API de gestión de errores basada en punteros del kernel de Linux.
- **Explicación:** en el desarrollo de Linux es habitual codificar valores de error (`errno`) directamente dentro de direcciones de puntero devueltas por funciones. Funciones inline como `IS_ERR()`, `PTR_ERR()` y `ERR_PTR()` permiten a Wazuh inspeccionar y manipular estos punteros de error sin depender de las librerías del kernel en espacio de usuario.

5.2.4 `/usr/include/linux/bpf_insn.h` y `/usr/include/bpf/bpf_insn.h`

- **Propósito:** proporcionar las estructuras e instrucciones base del ensamblador eBPF.
- **Explicación:** suministra las definiciones de registros e instrucciones esenciales de eBPF (como `BPF_MOV64_IMM` y `BPF_EXIT_INSN`). Al duplicar este archivo en ambas rutas (`/linux/` y `/bpf/`), se garantiza la compatibilidad tanto si el código fuente de Wazuh incluye el archivo mediante `<linux/bpf_insn.h>` como si lo hace mediante `<bpf/bpf_insn.h>`.

5.2.5 Parche a `/usr/include/linux/bpf.h` (`BPF_MAP_TYPE_ARENA`)

- **Propósito:** garantizar la definición del tipo de mapa eBPF `BPF_MAP_TYPE_ARENA`.
- **Explicación:** `BPF_MAP_TYPE_ARENA` es una constante agregada en versiones recientes del kernel Linux para asignación de memoria compartida en eBPF. Si la versión de las cabeceras del kernel instaladas en Alpine es anterior o no la incluye, esta inyección asegura que el código de Wazuh encuentre la constante (con valor 31) y compile sin arrojar errores de símbolo no declarado.

5.3 Paso 3 — Dependencias externas oficiales)

```
cd /root/wazuh/src
make deps TARGET=agent EXTERNAL_SRC_ONLY=yes
```

`EXTERNAL_SRC_ONLY=yes` pide código fuente (para compilarlo contra musl) en vez de binarios precompilados (que asumen glibc), esto reemplaza submódulos git, que durante las pruebas de la guía no funcionan dentro del contenedor Alpine.

5.3.1 3.1 — Excepción: `libbpf-bootstrap` necesita el paquete **PRECOMPILADO**

El programa eBPF del módulo FIM/whodata (`modern.bpf.c`) usa builtins CO-RE de Clang (`__builtin_preserve_access_index`) que Wazuh compila en un pipeline de CI dedicado por ar-

quitectura — no está pensado para compilarse en cualquier máquina. Si se deja como fuente, falla con errores de `implicit declaration` o `assignment... makes pointer from integer`. La solución es descartar la versión fuente y dejar que `make deps` traiga la versión precompilada (bytecode BPF, que no depende de glibc/musl):

```
rm -rf /root/wazuh/src/external/libbpf-bootstrap
cd /root/wazuh/src
make deps TARGET=agent
```

(sin `EXTERNAL_SRC_ONLY=yes` esta vez — como todo lo demás en `src/external` ya existe, `make deps` solo repuebla la carpeta que borramos, y esta vez con el binario ya compilado por Wazuh).

5.4 Paso 4 — Parche de OpenSSL para musl (bug de `strerror_r`)

El problema: `crypto/o_str.c` de OpenSSL decide qué variante de `strerror_r` usar según `#elif defined(_GNU_SOURCE)`. En glibc, definir `_GNU_SOURCE` cambia `strerror_r` para que devuelva `char *`. **musl nunca implementa esa variante** — su `strerror_r` siempre devuelve `int`, sin importar qué macros de feature test estén definidas. Dado que la compilación se realiza con `-D_GNU_SOURCE` globalmente, OpenSSL entra por la rama equivocada y falla de la siguiente manera:

```
error: assignment to 'char *' from 'int' makes pointer from integer without a cast
```

La corrección es parchar la condición para que en musl caiga en la rama correcta (la que sí funciona con `int`, que musl activa automáticamente vía `_POSIX_C_SOURCE`):

```
sed -i 's/#elif defined(_GNU_SOURCE)/#elif defined(_GNU_SOURCE) \&\& defined(__GLIBC__\n)/' /root/wazuh/src/external/openssl/crypto/o_str.c
```

5.5 Paso 5 — Compatibilidad de símbolos glibc faltantes en musl

Wazuh (concretamente `sysinfo/data_provider` y `libwazuhext`) llama a varias funciones que son extensiones exclusivas de glibc y que **musl nunca implementa**: la familia `_r` reentrante (`random_r`), los alias de dispositivo `gnu_dev_*`, toda la familia LFS64 (`open64`, `stat64`, `mkstemp64`, etc.), alias internos (`__strtok_r`, `__strdup`, `__rawmemchr`), `glob_pattern_p`, y el contexto de ejecución (`getcontext/setcontext/makecontext`, que musl omite a propósito por decisión de diseño, ya retirado de POSIX).

5.5.1 5.1 — Compilar `libexecinfo` (para `backtrace/backtrace_symbols`)

Alpine eliminó el paquete `libexecinfo-dev` de sus repositorios a partir de la versión 3.17 — hay que compilarlo desde el fork mantenido para musl:

```
git clone https://github.com/ronchaine/libexecinfo.git /root/libexecinfo
cd /root/libexecinfo
make
cp execinfo.h /usr/include/
cp libexecinfo.a /usr/lib/
cp libexecinfo.so.1 /usr/lib/
ln -sf /usr/lib/libexecinfo.so.1 /usr/lib/libexecinfo.so
```

5.5.2 5.2 — Compilar libucontext (para getcontext/setcontext/makecontext)

```
git clone https://github.com/kaniini/libucontext.git /root/libucontext
cd /root/libucontext
make ARCH=x86_64
make ARCH=x86_64 DESTDIR=/usr install
```

Arquitectura: ARCH=x86_64 está fijo en los dos comandos. Si el servidor Alpine corre en aarch64 (ver Requisitos previos), debe cambiarse por ARCH=arm64 en ambos antes de compilar.

Nota: el Makefile de libucontext ya trae /usr como prefijo interno fijo. Si con DESTDIR=/usr los archivos terminan en /usr/usr/lib/ y /usr/usr/include/ en vez de /usr/lib//usr/include/, muévanlos a mano:

```
cp -a /usr/usr/lib/. /usr/lib/
cp -a /usr/usr/include/. /usr/include/
rm -rf /usr/usr
```

libucontext exporta sus símbolos con el prefijo libucontext_* (libucontext_getcontext, etc.) más alias débiles (weak) sin prefijo. En la práctica, esos alias débiles no siempre se resuelven de forma confiable cuando quedan enterrados dentro de un enlace --whole-archive, así que el shim del siguiente paso los redirige explícitamente.

5.5.3 5.3 — El shim de compatibilidad completo (musl_compat.c)

Esta es la versión final, que ya incluye las redirecciones explícitas hacia libucontext (evita depender de sus símbolos débiles):

```
mkdir -p /root/wazuh/src/external/musl_compat
cat << 'EOF' > /root/wazuh/src/external/musl_compat/musl_compat.c
```

```

#define _GNU_SOURCE
#include <stdlib.h>
#include <stdarg.h>
#include <stdio.h>
#include <string.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <sys/mman.h>
#include <sys/sysmacros.h>
#include <unistd.h>

struct random_data { unsigned int seed; };

int srandom_r(unsigned int seed, struct random_data *buf) {
    if (!buf) return -1;
    buf->seed = seed ? seed : 1;
    return 0;
}

int random_r(struct random_data *buf, int32_t *result) {
    if (!buf || !result) return -1;
    *result = (int32_t) rand_r(&buf->seed);
    return 0;
}

int initstate_r(unsigned int seed, char *statebuf, size_t statelen, struct random_data
    *buf) {
    (void) statebuf; (void) statelen;
    return srandom_r(seed, buf);
}

unsigned int gnu_dev_major(unsigned long long dev) { return major(dev); }
unsigned int gnu_dev_minor(unsigned long long dev) { return minor(dev); }
unsigned long long gnu_dev_makedev(unsigned int maj, unsigned int min) { return
    makedev(maj, min); }

int open64(const char *path, int flags, ...) {
    mode_t mode = 0;
    if (flags & O_CREAT) {
        va_list ap; va_start(ap, flags);
        mode = (mode_t) va_arg(ap, int);
        va_end(ap);
    }
}

```

```

    return open(path, flags, mode);
}

int mkstemp64(char *tmpl) { return mkstemp(tmpl); }
int ftruncate64(int fd, off_t length) { return ftruncate(fd, length); }
int stat64(const char *path, struct stat *buf) { return stat(path, buf); }
int fstat64(int fd, struct stat *buf) { return fstat(fd, buf); }
int lstat64(const char *path, struct stat *buf) { return lstat(path, buf); }
off_t lseek64(int fd, off_t offset, int whence) { return lseek(fd, offset, whence); }
ssize_t pread64(int fd, void *buf, size_t count, off_t offset) { return pread(fd, buf,
    count, offset); }
ssize_t pwrite64(int fd, const void *buf, size_t count, off_t offset) { return pwrite(
    fd, buf, count, offset); }
void *mmap64(void *addr, size_t length, int prot, int flags, int fd, off_t offset) {
    return mmap(addr, length, prot, flags, fd, offset); }
int fseeko64(FILE *stream, off_t offset, int whence) { return fseeko(stream, offset,
    whence); }
off_t ftello64(FILE *stream) { return ftello(stream); }
FILE *fopen64(const char *path, const char *mode) { return fopen(path, mode); }
FILE *freopen64(const char *path, const char *mode, FILE *stream) { return freopen(
    path, mode, stream); }
FILE *tmpfile64(void) { return tmpfile(); }
char *__strtok_r(char *str, const char *delim, char **saveptr) { return strtok_r(str,
    delim, saveptr); }
char *__strdup(const char *s) { return strdup(s); }
void *__rawmemchr(const void *s, int c) { return memchr(s, c, (size_t)-1); }
int glob_pattern_p(const char *pattern, int quote) {
    for (const char *p = pattern; *p; p++) {
        if (*p == '\\' && quote) { if (*(p+1)) p++; continue; }
        if (*p == '*' || *p == '?' || *p == '[') return 1;
    }
    return 0;
}

/* Redirecciones explcitas para libucontext (no depender de sus alias weak) */
int libucontext_getcontext(void *ucp);
int libucontext_setcontext(const void *ucp);
int libucontext_swapcontext(void *oucp, const void *ucp);

__attribute__((visibility("default")))

```

```

int getcontext(void *ucp) {
    return libucontext_getcontext(ucp);
}
__attribute__((visibility("default")))
int setcontext(const void *ucp) {
    return libucontext_setcontext(ucp);
}
__attribute__((visibility("default")))
int swapcontext(void *oucp, const void *ucp) {
    return libucontext_swapcontext(oucp, ucp);
}
__asm__(
    ".text\n"
    ".global makecontext\n"
    ".type makecontext, @function\n"
    "makecontext:\n"
    "    jmp libucontext_makecontext\n"
);
EOF

gcc -c -fPIC /root/wazuh/src/external/musl_compat/musl_compat.c \
-o /root/wazuh/src/external/musl_compat/musl_compat.o

```

5.6 Paso 6 — Inyectar el shim en el Makefile de Wazuh

Por qué no basta con `LDFLAGS`: la receta de enlace de `libwazuhext.so` en el `Makefile` de Wazuh está escrita a mano (no es una regla implícita de Make) y no referencia `$(LDFLAGS)` en ningún lado, por lo que exportar la variable de entorno no tiene ningún efecto sobre ese enlace puntual. La receta real usa `${OSSEC_LIBS}` directamente (verificado con `grep -n -A20 "^${WAZUHEXT_LIB}"` sobre el `Makefile`), así que esa es la única variable por la que hay que inyectar nuestras librerías. **WAZUHEXT_OBJS no existe en ninguna receta del Makefile**, así que si se inyecta ahí, el objeto queda completamente ignorado y solo se resuelven los símbolos que ya traían `libucontext.a/libexecinfo.a` por su cuenta (`getcontext`, `backtrace`, etc.), mientras que los símbolos propios de `musl_compat.c` (`gnu_dev_*`, `__strtok_r`, `glob_pattern_p`, la familia `*64`) siguen indefinidos.

```
cd /root/wazuh/src
```

```
# 1. Restaurar el Makefile a su estado original (por si qued algñ sed
#     manual o una inyección previa en la variable equivocada)
git checkout Makefile

# 2. Inyectar el objeto de compatibilidad y las librerías necesarias
#     TODO en OSSEC_LIBS es la variable que la receta real usa
cat << 'EOF' >> Makefile
# Inyección de compatibilidad musl
OSSEC_LIBS += /root/wazuh/src/external/musl_compat/musl_compat.o /usr/lib/libucontext.
a /usr/lib/libexecinfo.a
EOF
```

5.6.1 Reconstruir libwazuhext.so y verificar

```
rm -f libwazuhext.so
make TARGET=agent CFLAGS="-D_GNU_SOURCE -I/usr/include -include /usr/include/linux/
bpf_insn.h -Wno-error=incompatible-pointer-types" libwazuhext.so

# Deben aparecer como 'T' (definido), no 'U' (indefinido)
nm -D libwazuhext.so | grep -E 'getcontext|setcontext|makecontext'
```

5.7 Paso 7 — Compilación completa del agente (con CMAKE_OPTS para los sub-builds de CMake)

sysinfo/data_provider y syscollector son sub-proyectos de **CMake** anidados dentro del Makefile principal — CMake no lee LDFLAGS del entorno, así que sus flags de enlace hay que pasarlos por CMAKE_OPTS (CMAKE_EXE_LINKER_FLAGS / CMAKE_SHARED_LINKER_FLAGS), envolviendo las librerías estáticas en --whole-archive/--no-whole-archive para forzar que se incluyan todos sus símbolos:

```
cd /root/wazuh/src
make TARGET=agent \
    CFLAGS="-D_GNU_SOURCE -I/usr/include -include /usr/include/linux/bpf_insn.h -Wno-
    error=incompatible-pointer-types" \
    CMAKE_OPTS='-DCMAKE_EXE_LINKER_FLAGS="/root/wazuh/src/external/musl_compat/
    musl_compat.o -Wl,--whole-archive /usr/lib/libexecinfo.a /usr/lib/libucontext.a -
    Wl,--no-whole-archive" -DCMAKE_SHARED_LINKER_FLAGS="/root/wazuh/src/external/
    musl_compat/musl_compat.o -Wl,--whole-archive /usr/lib/libexecinfo.a /usr/lib/
    libucontext.a -Wl,--no-whole-archive"'
```

Después de ejecutar este paso y esperar algunos minutos a la compilación del agente, se debería obtener el siguiente resultado:

```
Compiler:
CFLAGS -pthread -Iexternal/pacman/lib/libalpm/ -Iexternal/libarchive/libarchive -Wl,--start-group -Iexternal/audit-userspace/lib -g -DDEBUG -O2 -DOSSECHIDS -DUSER="wazuh" -DGROUPGLOBAL="wazuh" -Dlinux -DINOTIFY_ENABLED -D_XOPEN_SOURCE=600 -D_GNU_SOURCE -DALPINE -DIMAGE_TRUST_CHECKS=1 -DCA_NAME='DigiCert Assured ID Root CA' -DENABLERSYS -DENABLERCISCAT -DENABLERAUDIT -DCLIENT -pipe -Wall -Wextra -std=gnu99 -I./ -I./headers/ -Iexternal/openssl/include -Iexternal/cJSON/ -Iexternal/libyaml/include -Iexternal/curl/include -Iexternal/msgpack/include -Iexternal/bzip2/ -Ishared_modules/common -Ishared_modules/dbsync/include -Ishared_modules/rsync/include -Iwazuh_modules/syscollector/include -Idata_provider/include -Iexternal/libpcap2/include -Iexternal/rpm/build/output/include -Isyscheckd/include -Ishared_modules/router/include -Ishared_modules/content_manager/include -Iwazuh_modules/vulnerability_scanner/include -I./shared_modules/ -D_GNU_SOURCE -I/usr/include -include /usr/include/linux/bpf_insn.h -Wno-error=compatible-pointer-types
LDFLAGS -Wl,-rpath,../lib -pthread -lrt -ldl -O2 -Lshared_modules/dbsync/build/lib -Lshared_modules/rsync/build/lib -Lwazuh_modules/syscollector/build/lib -Ldata_provider/build/lib -Lsyscheckd/build/lib
LIBS -lintl -lrt -ldl -lm /root/wazuh/src/external/musl_compat/musl_compat.o /usr/lib/libucontext.a /usr/lib/libexecinfo.a
CC gcc
MAKE make
make[1]: Leaving directory '/root/wazuh/src'

Done building agent

30120e01195c:~/wazuh/src#
```

Figura 1: Resultado esperado tras compilar el agente en el Paso 7

Una vez en este punto, siempre que se quiera volver a compilar el agente se reducirá el tiempo de espera, porque `make` solo recompila lo que cambió: las dependencias de `src/external` (OpenSSL, libbpf-bootstrap, etc.) y los sub-builds de CMake ya quedaron resueltos y no se vuelven a construir desde cero. Si solo se modifica un archivo fuente propio de Wazuh, basta con repetir el comando de este paso — no hace falta repetir los Pasos 3 a 6.

5.8 Paso 8 — Instalar el agente compilado

```
cd /root/wazuh
./install.sh
```

Es un script interactivo: debe elegirse **agent** (no manager) cuando pregunte el tipo de instalación, y aceptarse el directorio destino por defecto (`/var/ossec`). Como todo ya se compiló en `src/`, este paso solo copia los binarios a su ubicación final — no vuelve a compilar nada.

5.9 Paso 9 — Registrar el agente

Registrar el agente contra el manager (genera `client.keys`):

```
/var/ossec/bin/agent-auth -m IP_DEL_MANAGER
```

Si este comando falla, ver la sección **Casos de error**, al final de esta guía.

5.10 Paso 10 — Verificar el estado del agente

En algunos casos al ejecutar el paso 9 el agente ya queda activo y registrado en wazuh dashboard, en caso contrario se debe iniciar manualmente de la siguiente manera:

```
/var/ossec/bin/wazuh-control stop
rm -f /var/ossec/var/run/*.pid
/var/ossec/bin/wazuh-control start
```

Se deben ver los 5-6 demonios (`wazuh-execd`, `wazuh-agentd`, `wazuh-logcollector`, `wazuh-syscheckd`, `wazuh-modulesd`, etc.) arrancar uno tras otro sin el mensaje `Process not used by Wazuh, removing`. Confirmar:

```
ps aux | grep wazuh
tail -50 /var/ossec/logs/ossec.log
```

Y por último, confirmar en el **manager** que el agente ya no aparece como *never connected* sino como **active**. Si el agente no arranca, se recomienda ver la siguiente sección de **CASOS DE ERROR**.

6 CASOS DE ERROR

6.1 Si el Paso 3.1 falla — Cabeceras faltantes de `bpftool` bootstrap

Nota: si se siguió el Paso 3.1 tal como está escrito (dejando que `make deps` traiga la versión **precompilada** de `libbpf-bootstrap`), este caso de error **no debería aplicar** — la carpeta `build/bpftool/` simplemente no se genera con el binario precompilado, así que un `find -name "bpf.h"` no encontrará nada, y eso es lo esperado, no una falla. Este caso queda documentado solo para quien fuerce la compilación de `libbpf-bootstrap` **desde fuente** en vez de usar el precompilado.

Al compilar `libbpf-bootstrap` **desde fuente** puede fallar con:

```
fatal error: bpf/bpf.h: No such file or directory
```

Es un bug documentado del propio `bpftool` (parche del kernel de nov. 2021): el Makefile de la versión "bootstrap" no instala las cabeceras de `libbpf` en la ruta que su propio `main.c` espera, aunque sí las genera en otro lugar del árbol de build. La corrección, solo para ese escenario:

```
find /root/wazuh/src/external/libbpf-bootstrap -name "bpf.h"
```

```
cp /root/wazuh/src/external/libbpf-bootstrap/build/bpftool/bootstrap/libbpf/include/
bpf/*.h /usr/include/bpf/
```

6.2 Si la IP y el puerto no son correctos

Si durante la instalación se agregó mal la IP o el puerto del manager, o el agente quedó configurado con otro protocolo de conexión, estos valores se pueden corregir manualmente en el archivo de configuración del agente, sin necesidad de reinstalar nada:

Editar `/var/ossec/etc/ossec.conf`, dentro de la sección `<client>`:

```
<server>
  <address>IP_DEL_MANAGER</address>
  <port>1514</port>
  <protocol>tcp</protocol>
</server>
```

6.3 Fallo al obtener la llave del Wazuh manager

Si al ejecutar el Paso 9 el registro falla, conviene revisar en este orden:

1. **Conectividad y puertos.** El puerto de **enrollment** (registro) es el **1515**; el puerto de **conexión persistente** (Paso 10, una vez arrancado el agente) es el **1514**. Son dos puertos distintos — que uno responda no garantiza que el otro también esté abierto en el firewall. Antes de sospechar de la IP, conviene probar conectividad directa al puerto de registro:

```
apk add --no-cache netcat-openbsd
nc -zv IP_DEL_MANAGER 1515
```

Un timeout apunta a firewall/red (en labs con pfSense segmentado por zonas, revisar las reglas entre la zona de este contenedor y la del manager); un "connection refused" apunta a que `wazuh-authd` no está habilitado/corriendo en el manager. Sin conectividad en este puerto, ninguna de las opciones siguientes va a funcionar.

2. **Nombre de agente duplicado.** Si hubo intentos fallidos previos, puede que ya exista un agente registrado con ese nombre. Desde el dashboard, en **Agents management** → **Summary**, puede buscarse el nombre y eliminarlo antes de reintentar, o registrarse con un nombre distinto agregando `-A <nombre>` al comando del Paso 9:

```
/var/ossec/bin/agent-auth -m IP_DEL_MANAGER -A nombre-alternativo
```

3. **Obtener la llave manualmente vía la API de Wazuh**, si `agent-auth` sigue fallando pero el manager sí responde. Esto es útil porque separa el problema de red del problema de generación de la llave: si la API logra entregar una llave, el problema estaba en `wazuh-authd`/puerto 1515, no en el manager en general.

Desde el dashboard: menú lateral → **Server management** → **Dev Tools**, que lleva a la siguiente interfaz para interactuar con la API de Wazuh directamente:

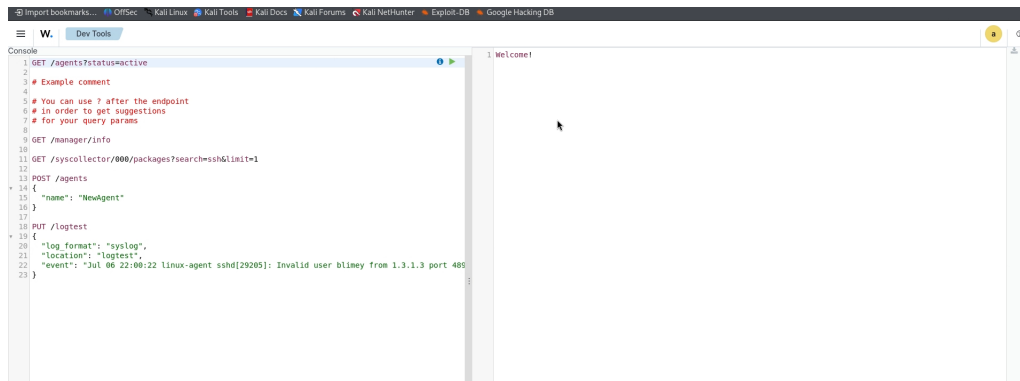


Figura 2: Interfaz de Dev Tools en el dashboard de Wazuh

El mismo flujo puede hacerse desde línea de comandos con `curl` — autenticarse contra la API (puerto **55000**/**TCP** por defecto) para obtener un token, y usar ese token para pedir la llave del agente:

```
# 1. Autenticarse y obtener un token (usuario/clave por defecto de la API: wazuh/
wazuh,
#     que debe cambiarse si el manager ya tiene credenciales distintas
#     configuradas)
TOKEN=$(curl -u wazuh:wazuh -k -X POST \
  "https://IP_DEL_MANAGER:55000/security/user/authenticate?raw=true")

# 2. Registrar el agente por API (si an no existe) y obtener su id
curl -k -X POST "https://IP_DEL_MANAGER:55000/agents" \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"name": "nombre-del-agente"}'

# 3. Con el id devuelto en el paso anterior, pedir la llave en texto plano
curl -k -X GET "https://IP_DEL_MANAGER:55000/agents/<AGENT_ID>/key?raw=true" \
  -H "Authorization: Bearer $TOKEN"
```

La llave que devuelve el último comando se importa **en el agente**, no en el manager, con:

```
/var/ossec/bin/manage_agents -i <LLAVE_OBTENIDA>
```

Este método reemplaza a `agent-auth` para ese agente puntual; luego se continúa con el Paso 10 (arrancar el agente) normalmente.

6.4 Errores arrancando el agente (Paso 11)

El problema: `wazuh-control` verifica si un proceso sigue vivo con `ps -p $PID`. BusyBox (el `ps` de Alpine) no soporta ese flag de la misma forma que el `ps` de GNU/coreutils, así que el chequeo falla, el script concluye — incorrectamente — que el proceso no es de Wazuh, lo da de baja, y **aborta el resto de la secuencia de arranque** sin seguir con los demás demonios (típicamente se corta justo después de `wazuh-execd`, y `wazuh-agentd` —el que mantiene la conexión persistente con el manager— nunca llega a arrancar; el agente queda registrado pero en estado *never connected*).

La corrección es reemplazar `ps -p $PID` por `kill -0 $PID`, que verifica la existencia del proceso sin depender de flags no estándar de `ps`:

```
sed -i 's/ps -p $j > \dev\/null 2>&1/kill -0 $j > \dev\/null 2>\&1/' /var/
ossec/bin/wazuh-control
sed -i 's/ps -p ${pid} > \dev\/null 2>&1/kill -0 ${pid} > \dev\/null 2>\&1/' /
var/ossec/bin/wazuh-control

# Confirmar que ambas quedaron reemplazadas (deben aparecer como kill -0, no ps -
p)
grep -n "ps -p\|kill -0" /var/ossec/bin/wazuh-control
```

7 Comandos de utilidad

Además de los comandos ya usados paso a paso, estas herramientas ayudan a diagnosticar problemas de símbolos, librerías y binarios en cualquier punto de la compilación — y son las mismas que se usaron para encontrar los errores documentados en esta guía.

7.0.1 Inspeccionar símbolos: `nm`

`nm` lista los símbolos de un objeto (`.o`) o librería (`.a/.so`) junto con una letra que indica su tipo. Las tres que más importan al depurar errores de enlazado (`undefined reference`) son:

- **T** (o **t**): símbolo **definido** en la sección de código (*text*). Significa que esa función sí está implementada y exportada en ese archivo.
- **W** (o **w**): símbolo **débil** (*weak*). Puede ser reemplazado por otra definición del mismo nombre sin generar error de símbolo duplicado — así es como el shim `musl_compat.c` puede coexistir con símbolos que `musl` sí define parcialmente.
- **U**: símbolo **no definido** (*undefined*) en ese archivo — se espera que otro objeto o librería lo resuelva al enlazar. Un **U** que nunca se resuelve en ningún archivo es exactamente lo que produce un error `undefined reference` al final de la compilación.

```
# Ver si un smbolo puntual esta definido (T/W) o pendiente (U) en un .a/.so
nm -D /usr/lib/libwazuhext.so | grep getcontext

# Lo mismo sobre un objeto compilado a mano
nm src/external/musl_compat/musl_compat.o | grep -i strtok_r

# Listar todos los simbolos no resueltos (U) de un binario o libreria
nm -u /var/ossec/bin/wazuh-agentd
```

7.0.2 Identificar binarios y buscar texto: `file` y `strings`

```
# Confirmar arquitectura, si es dinamico o estatico, y si tiene simbolos de
# depuracion
file /var/ossec/bin/wazuh-agentd

# Buscar una cadena (por ejemplo una version o una ruta) dentro de un binario
strings /usr/lib/libwazuhext.so | grep -i openssl
```

Nota: Copiar y pegar comandos directamente de un PDF hacia una terminal introduce errores silenciosos que no siempre son obvios a simple vista y en la mayoría de los casos se recomienda a los estudiantes verificar cada salida de los comandos antes de seguir al siguiente paso.

HISTORIAL DE REVISIONES

Fecha	Autor(es)	Versión	Referencia de Cambios
22/08/2026	Samuel Rodríguez T.	1.0	Versión inicial de la guía

REVISOR(ES)

Nombre	Cargo	Institución
Julián Contreras	Asistente graduado del curso de Ciberseguridad	Universidad de los Andes
Carlos Lozano	Profesor Asociado del Departamento de Ingeniería de Sistemas y Computación	Universidad de los Andes